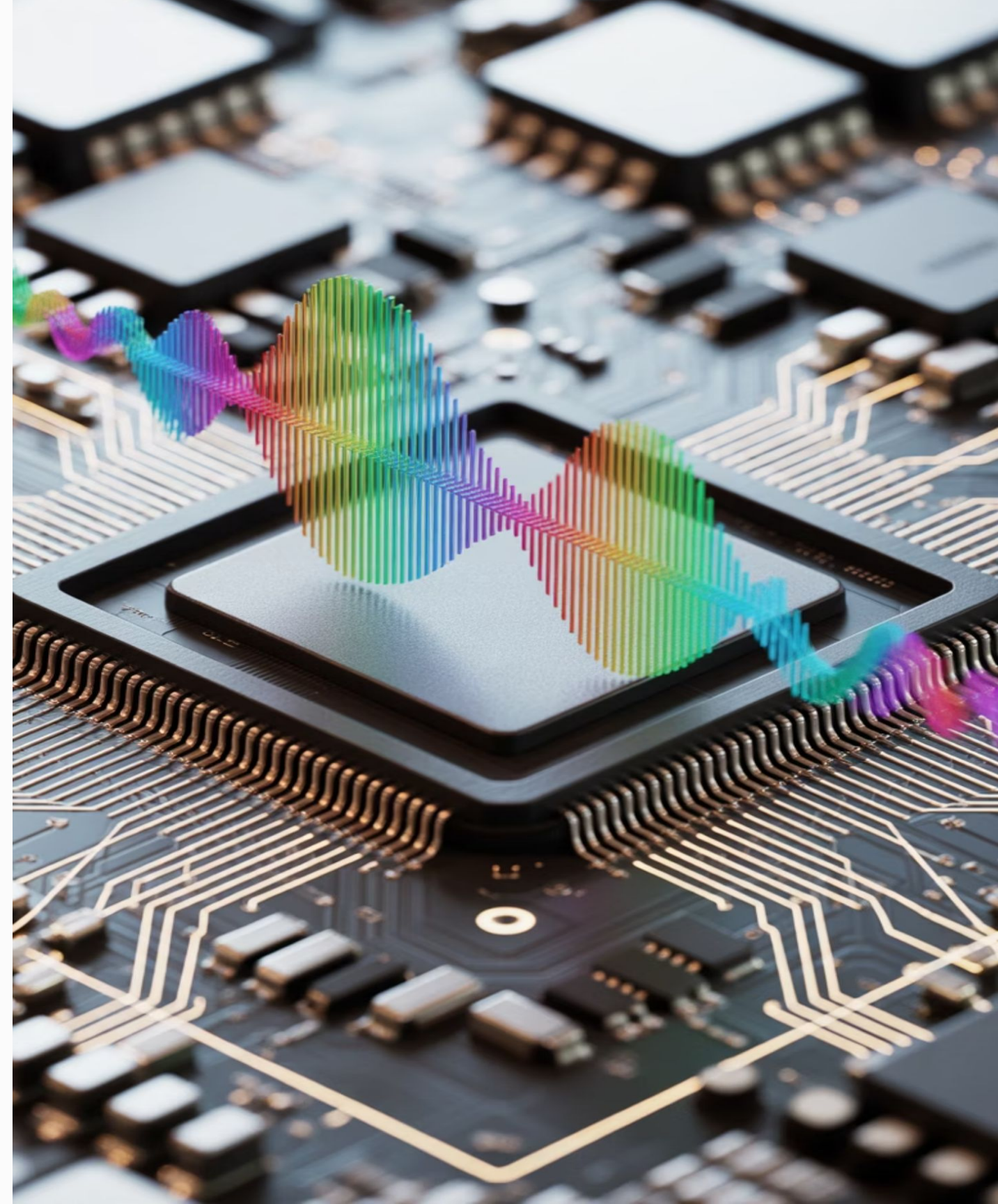
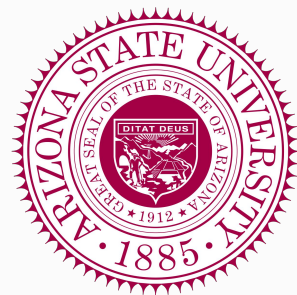


Lightweight Keyword Spotting Using MFCC and Energy Features for Embedded Systems

EEE 598 - Speech and Audio Processing Final Project

Saif Elsaady, Yousef El Sheikha | December 2025

Instructor: Dr. Berisha



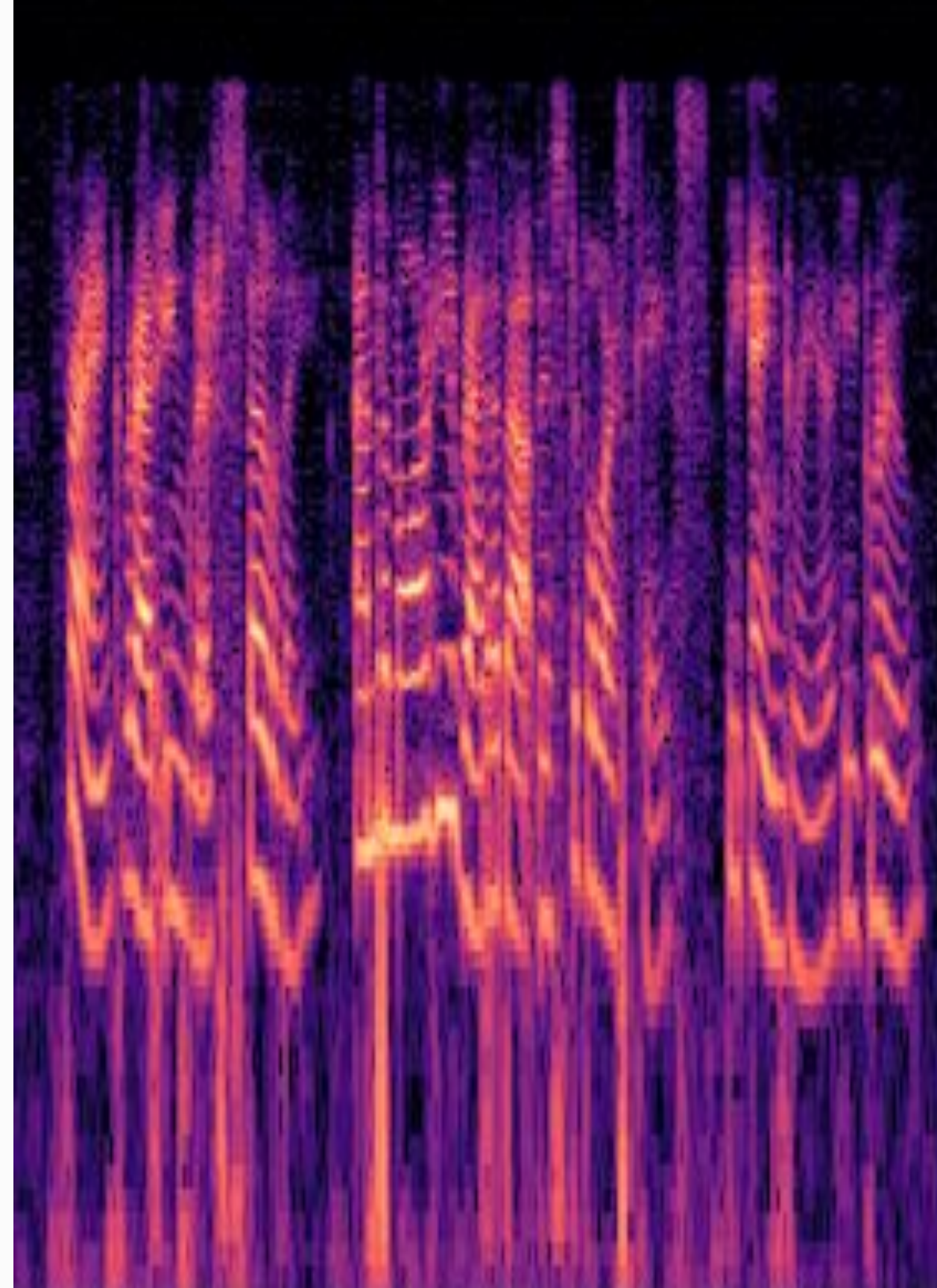
The Need for Efficient Lightweight On-Device Speech Recognition

Current Landscape Challenges

- **Privacy Risks:** Sending audio to the cloud for processing is a security risk.
- **Too Big:** Deep learning models (100MB+) are too large and slow for cheap, battery-powered chips.

Our Classical DSP Solution

- **100% Private:** Runs entirely on the device.
- **Tiny & Fast:** Using efficient math (Energy, ZCR, MFCCs), we process keywords in **<10ms**.
- **Ultra-Light:** The final model is **<1MB**, perfect for low-power embedded systems.



Project Objectives & Requirements

01

Full Pipeline Implementation

We built a complete speech recognition system from scratch in MATLAB, without pre-trained models, using core DSP methods.

03

Comparative Classifier Study

k-NN: Simple "nearest neighbor" model

SVM: Advanced "best boundary" model (RBF)

- Tuned all settings for best performance

02

Multi-Domain Feature Extraction

We extracted features from both time and frequency domains, including short-time energy, zero-crossing rate, and MFCCs.

All features were combined into a 30-dimensional feature vector.

04

Performance Metrics

Target: >75% accuracy (5 classes)

Analysis: Per-word accuracy

Test: Noise robustness (Inf to -5 dB SNR)

Google Speech Commands Dataset V2

Class Distribution

Keyword	Train	Val	Test	Total
Yes	3,200	400	405	4,005
No	3,000	375	394	3,769
Stop	3,100	387	387	3,874
Go	3,150	393	388	3,931
Silence/ Unknown	1,600	200	200	2,000

*Counts shown reflect our processed dataset after resampling, trimming, and splitting.

Dataset Statistics

Our dataset consists of ~24,000 one-second clips, notable for its diversity in speakers, accents, microphones, and recording environments.



Data Split: 80% train, 10% validation, 10% test (stratified by class)

Four-Stage Processing Pipeline Overview



Input & Preprocessing

Resample, normalize,
trim silence of .wav files.



Framing

20ms windows @ 10ms
hop



Features

Energy, ZCR, MFCCs
(extraction, per frame)



Classification

Inference using trained
k-NN or SVM classifier by
assigning label

Memory Usage

- ~30-D feature vector (~120 bytes/clip).
- Model size <1 MB (supports embedded deployment).

Processing Flow

- Batch processing for training
- Real-time (<10 ms) per-clip inference

Design Philosophy

- Modular design enables swapping/ablation of preprocessing, feature extraction, and classifiers.

Methods: Audio Standardization Pipeline



Format Standardization

Input: 16 kHz Google Speech clips (pipeline can resample other rates)

Output: Uniform 16 kHz using polyphase resampling



Amplitude Normalization

Method: Peak normalization to $[-1, 1]$

Purpose: Consistent dynamic range across samples



Silence Trimming Algorithm

Frame-based energy

Threshold: -45 dB below peak energy

Minimum retention: 120ms (preserves short utterances)



Length Standardization

Target: Exactly 16,000 samples (1.0 second)

Short clips: zero-pad at the end to reach 1.0 s

Long clips: Crop around the main speech region to 1.0 s

Methods: Mel-Frequency Cepstral Coefficients (MFCCs)

1. Analyze the Sound

- a. We break the sound into tiny 20ms slices to see the pitch and loudness in each one.

2. Mimic Human Perception (Mel Filterbank)

- a. We group those pitches using 26 special filters that mimic how the human ear works (focusing more on low sounds).

3. Adjust for Loudness (Log Compression)

- a. We then take the log of those 26 energy values. This models human loudness perception and stabilizes dynamic range. This is just like our ears, which are much better at hearing differences in quiet sounds than in very loud ones.

4. Pack the Features (DCT → MFCCs)

A discrete cosine transform compresses the 26 energies into 13 MFCCs:

- a. **MFCC 0:** Represents the average log energy across the filterbank (overall spectral envelope).
- b. **MFCCs 1-12:** Describes the unique spectral shape, useful for distinguishing words (helps our model tell "yes" from "no" for example).

Summary: MFCCs turn complex audio waves into a small, simple list of numbers that describe the sound's unique shape.

Methods: Time-Domain Feature Extraction

General Purpose: Time-domain features complement MFCCs by capturing amplitude and voicing patterns that spectral features alone do not preserve.

Short-Time Energy (STE)

- **What it measures:** The "loudness" or intensity of the signal over time.
- **Why it is important:** It helps represent the "pulse" of the word.
- **How we use it:**
 - **Word Boundaries/Bursts:** It helps the model detect exactly where a word starts and ends (distinguishing speech from silence).
 - **Syllable Detection:** It captures the amplitude envelope, helping distinguish words with strong bursts (like "go") from ones with softer onsets.

Zero-Crossing Rate (ZCR)

What it measures: How frequently the audio signal flips between positive and negative values.

Why it is important: It acts as a "rough frequency" detector without needing complex math.

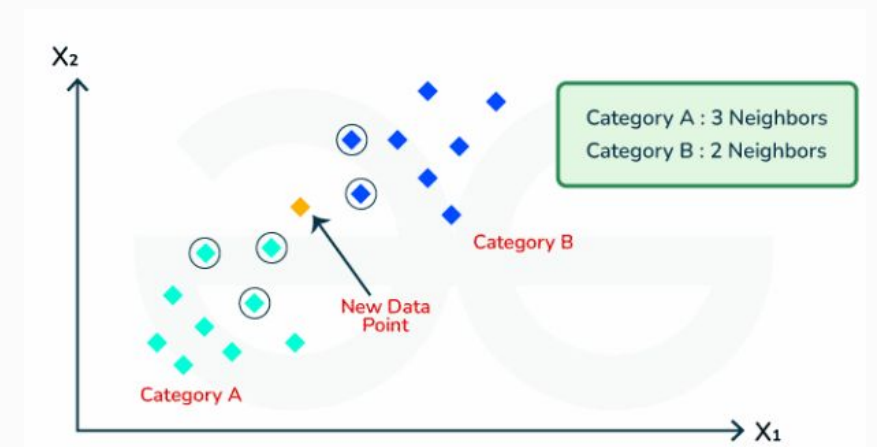
How we use it ("Texture" Detector):

- **Voiced vs. Unvoiced:** It easily separates "noisy" sounds from "smooth" sounds.
- **Critical for our Keywords:**
 - **High ZCR:** Detects fricatives like the 's' in "stop" or "yes."
 - **Low ZCR:** Detects vowels like the 'o' in "no" or "go."

Comparative Analysis: kNN vs SVM

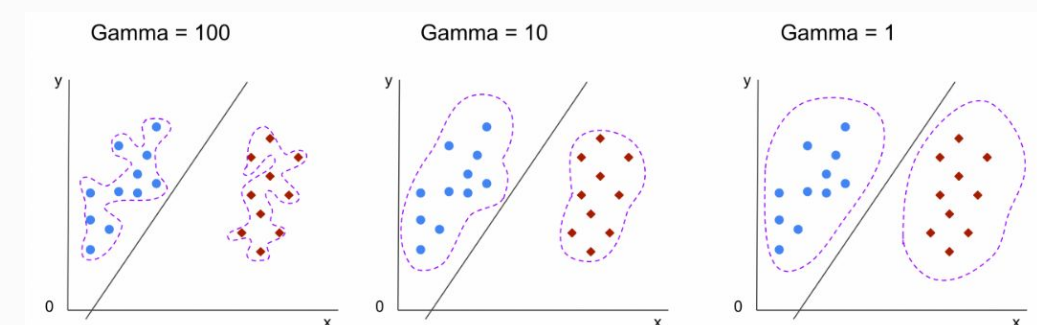
1. k-Nearest Neighbors (k-NN)

- **How it Works:** Finds the "closest" sound fingerprints in the training data and takes a "majority vote" to decide.
- **Best Validation Setting:** We tested 4 different settings ($K=1, 3, 5, 7$) and found that **$K=1$** gave the best validation accuracy at **79.53%**.
- **Pros:** It's a simple idea and easy to set up.
- **Cons:** It's slow and has to store the *entire* training dataset to work.



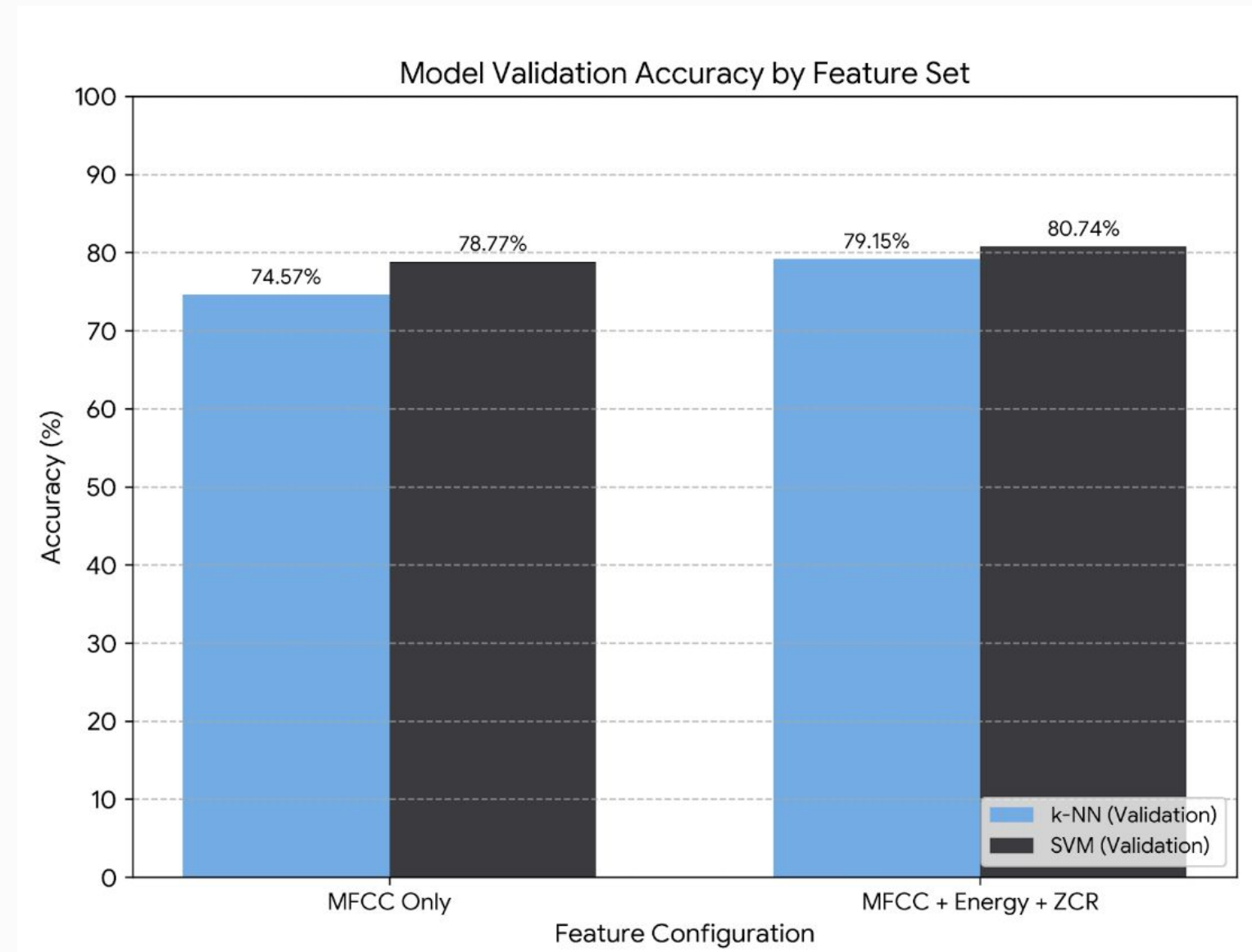
2. Support Vector Machine (SVM)

- **How it Works:** Uses an RBF kernel to find the best possible non-linear boundary to separate the keywords.
- **Best Validation Setting:** After testing 12 different combos, the best settings gave us **83.19%** accuracy.
- **Pros:** It's more accurate, faster at inference, better at guessing on new data, and the final model is small.
- **Cons:** It's more complex to train and tune and longer training time.



Feature Ablation Study Results

- **Comparison:** MFCC Only vs. MFCC + Energy + ZCR.
- **Result:** Adding time-domain features (Energy/ZCR) improved accuracy across the board.
- **Model Choice:** SVM consistently beat k-NN.
- **Final Decision:** Selected **SVM with Combined Features (80.74%)**.



Comparing Feature Sets: MFCC-only vs MFCC+Energy+ZCR (SVM)

MFCC-only (Acc=84.1%)						
True	yes	385	5	7	8	
	no	19	293	7	75	
	stop	35	14	323	15	
	go	18	67	12	291	
	silence					200

95.1%	4.9%
74.4%	25.6%
83.5%	16.5%
75.0%	25.0%
100.0%	

84.2%	77.3%	92.6%	74.8%	100.0%
15.8%	22.7%	7.4%	25.2%	
yes	no	stop	go	silence
Predicted				

MFCC+Energy+ZCR (Acc=84.7%)									
True	yes	384	9	7	5		94.8%	5.2%	
	no	19	303	4	68		76.9%	23.1%	
	stop	41	8	316	22		81.7%	18.3%	
	go	18	63	7	300		77.3%	22.7%	
	silence				1	199	99.5%	0.5%	
		83.1%	79.1%	94.6%	75.8%	100.0%			
		16.9%	20.9%	5.4%	24.2%				
		yes	no	stop	go	silence			
		Predicted							

Key Observations:

- MFCC-only already performs very strongly (carry most of the discriminative power).
- Adding Energy + ZCR gives a small accuracy boost (~+0.6%).
- ZCR improves detection of fricatives (“s” in stop, yes).
- Energy helps with bursty onsets (ex: go).
- MFCC+Energy+ZCR shows slightly fewer confusions overall (more stable across word types).

Best Model: Confusion Matrix Summary and Analysis

MFCC+Energy+ZCR SVM Performance - 84.67% Accuracy

True Class	Precision	Recall	F1-Score	Errors
Yes	83.1%	94.8%	88.6%	→ stop (41)
No	79.1%	76.9%	78.0%	→ go (63)
Stop	94.5%	81.7%	87.6%	→ yes (41)
Go	75.8%	77.3%	76.5%	→ no (67)
Silence/Unknown	100%	99.5%	99.7%	Very stable

Error Pattern Analysis

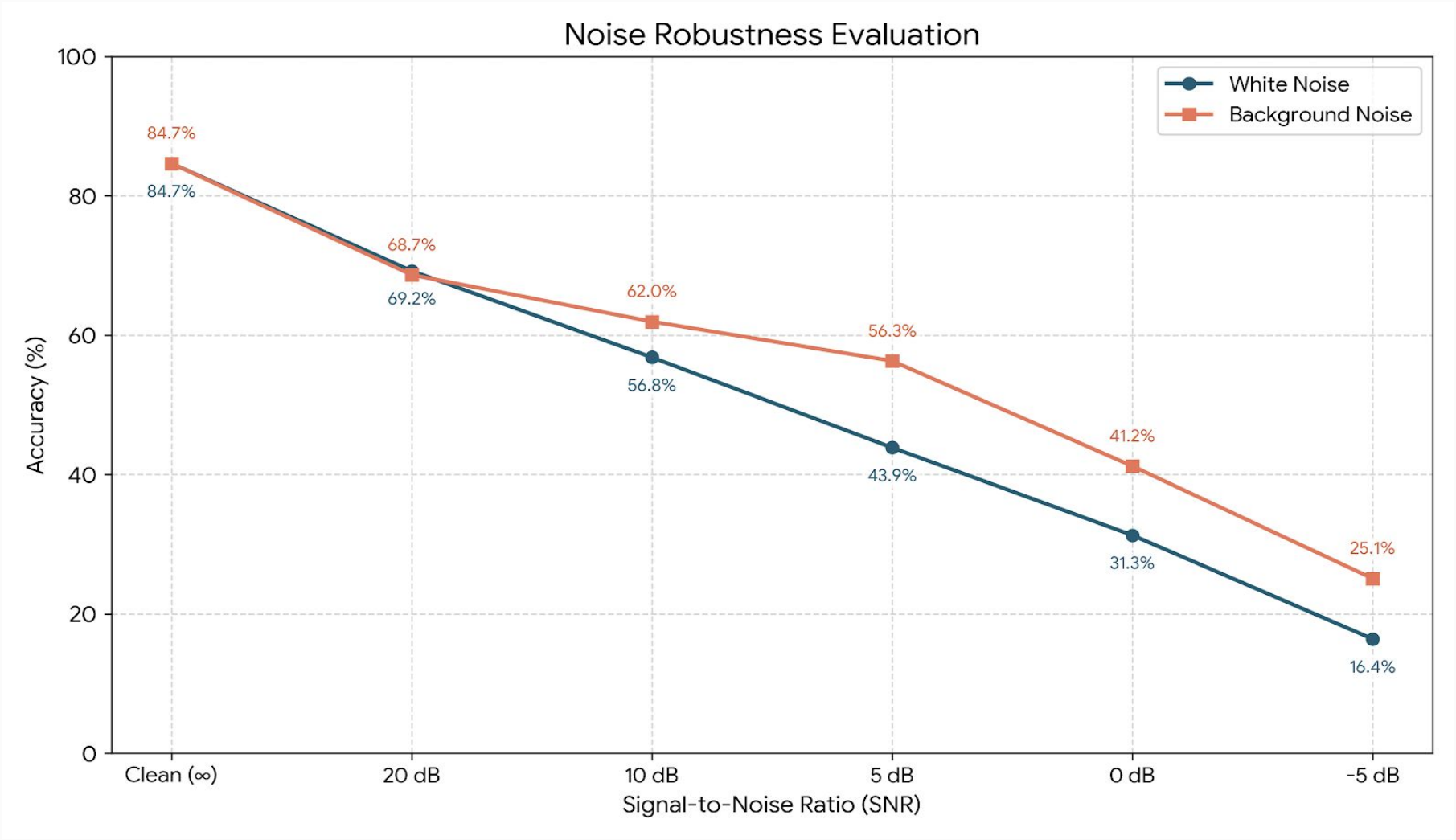
Highest confusion: "no" ↔ "go" (similar vowel structure and duration)

Best performance: "Silence/unknown" nearly perfect (distinct energy profile)

- Front-loaded words ("yes", "stop") show cross-confusion
- Overall: 1,603/1,894 correct predictions

Noise Robustness Evaluation

System Performance Under Adverse Conditions



White	SNR	Inf dB	->	Acc = 84.67%
White	SNR	20 dB	->	Acc = 69.22%
White	SNR	10 dB	->	Acc = 56.82%
White	SNR	5 dB	->	Acc = 43.86%
White	SNR	0 dB	->	Acc = 31.29%
White	SNR	-5 dB	->	Acc = 16.40%
BG	SNR	Inf dB	->	Acc = 84.67%
BG	SNR	20 dB	->	Acc = 68.66%
BG	SNR	10 dB	->	Acc = 61.95%
BG	SNR	5 dB	->	Acc = 56.31%
BG	SNR	0 dB	->	Acc = 41.21%
BG	SNR	-5 dB	->	Acc = 25.08%

We tested our model by adding both white noise and real background noise to all 1,894 test clips at various SNR levels. The model's accuracy degraded gracefully, staying above 50% even up to 10 dB of noise. We also found it was slightly better at handling real background noise than pure white noise.

Project Summary

84.67%

Accuracy Achieved

Exceeded 75% target by
9.67%

30

Feature Dimensions

Compact representation
(mean & std of MFCCs (26)
+ energy (2) + ZCR (2))

<10ms

Inference Time

Real-time capable (~9.6 ms
for 1-second audio)

<1MB

Model Size

Embedded-friendly SVM model

Lessons Learned

We proved that a classical DSP pipeline, built from scratch without deep learning, can achieve strong accuracy on small vocabularies. By optimizing features (MFCCs) and models (SVM), we created a system that is accurate, real-time capable, and perfect for tiny, resource-constrained chips.

Future Work & Real-World Applications

Proposed Enhancements

01

Advanced Features

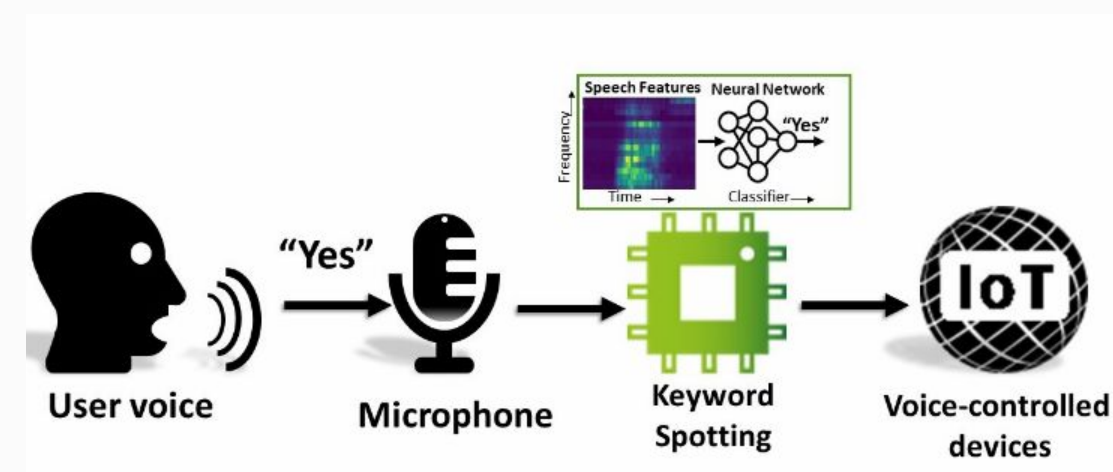
- Track how sounds change over time (not just the static sound).
- Add "Pitch" to better understand vowel sounds and tones.
- Speaker gender identification

This type of lightweight KWS design is commonly used in hands-free voice interfaces for smart home devices, wearables, IoT sensors, and other embedded systems. Our project demonstrates the core signal-processing foundations used in these systems.

02

Algorithmic Improvements

- Combine our SVM with other models and have them "vote" for the best answer.
- Add a smart noise-cancelling step before we process the audio.



References

1. Kaladin Stormblessed, “Google Speech Commands V2,” Kaggle.com, 2023. <https://www.kaggle.com/datasets/sylkaladin/speech-commands-v2?select=bed>.
2. A. de Cheveigné and H. Kawahara, “YIN, a fundamental frequency estimator for speech and music,” Journal of the Acoustical Society of America, vol. 111, no. 4, pp. 1917–1930, 2002.
3. G. Fant, “The LF-model revisited. Transformations and frequency domain analysis,” Speech Transmission Laboratory Quarterly Progress and Status Report, vol. 26, no. 2–3, pp. 119–156, 1985.
4. S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 28, no. 4, pp. 357–366, 1980.
5. P. Warden, “Speech Commands: A dataset for limited-vocabulary speech recognition,” arXiv preprint arXiv:1804.03209, 2018.
6. Y. Zhang, N. Suda, and L. Lai, “Hello Edge: Keyword spotting on microcontrollers,” arXiv preprint arXiv:1711.07128, 2017.

Acronyms

DSP – Digital Signal Processing
MFCC – Mel-Frequency Cepstral Coefficients
ZCR – Zero-Crossing Rate
STE – Short-Time Energy
KWS – Keyword Spotting
SVM – Support Vector Machine
k-NN – k-Nearest Neighbors
RBF – Radial Basis Function
DCT – Discrete Cosine Transform
SNR – Signal-to-Noise Ratio
IoT – Internet of Things
STFT – Short-Time Fourier Transform
VAD – Voice Activity Detection
CPU – Central Processing Unit
F1 – F1 Score
TP – True Positive
FN – False Negative
FP – False Positive
TN – True Negative
MATLAB – Matrix Laboratory
WAV – Waveform Audio File Format
FB – Filterbank
Hz – Hertz
dB – Decibel
ms – Millisecond
RAM – Random Access Memory
ROM – Read-Only Memory
ROC – Receiver Operating Characteristic
AUC – Area Under the Curve

Thank You !

